ML Primer for This Project (Math Background Assumed)

1. What a neural network actually is

Forget the brain metaphor for a second — a neural network is just a big composite function made of matrix multiplications and simple nonlinear functions, stacked in layers:

```
output = f_n(... f_2(f_1(W_1 · input + b_1)) ...)
```

Each `W` is a matrix of numbers ("weights") and `b` is a vector ("bias"). `f` is a simple nonlinear function (like ReLU: `max(0, x)`). "Training" means adjusting the numbers in every `W` and `b` so the function's output gets closer to what you want.

2. How training actually works (you know this math)

- **Loss function:** a number measuring how wrong the model's output is (e.g. cross-entropy for next-word prediction). Lower = better.
- **Gradient descent:** you take the derivative of the loss with respect to every single weight (that's what "gradient" means here — a vector of partial derivatives), then nudge each weight slightly in the direction that reduces loss. This is just multivariable calculus, done automatically by the framework (PyTorch) via backpropagation (repeated chain rule).
- **Learning rate (LR):** the step size of that nudge. Too big → overshoots and never converges. Too small → painfully slow.
- **Optimizer (Adam):** a smarter version of plain gradient descent that keeps a running average of past gradients to adjust step size per-parameter. This is why "optimizer state reset" (a Tier 2 fallback method) means something specific — Adam has memory beyond just the weights themselves.

3. What "training a language model" specifically means

The model's job: given a sequence of words/tokens, predict the next one. You feed it text, it guesses the next token, you compare its guess to the actual next token (that's the loss), and adjust weights to make it guess better next time. Repeat billions of times. "Pretraining" just means this basic next-token-prediction training, before any fine-tuning for a specific task.

4. Tokens

Text gets chopped into "tokens" — not quite whole words, not quite letters (e.g. "unhappiness" might become "un" + "happi" + "ness"). The model never sees raw text, only sequences of token IDs (integers). "10M words" roughly corresponds to a somewhat larger number of tokens.

5. Embeddings

Before the model can do any math on a token, that token (just an integer ID) needs to

become a vector of numbers it can compute with. The **embedding layer** is a lookup table: token ID → vector (e.g. a 256 or 512-length list of numbers). This vector starts as random noise and gets shaped by training to encode meaning. This is why "resetting the embedding layer" (your intervention #1) is such a big deal — it wipes out everything the model has learned about how to represent every single token, forcing it to relearn those representations from scratch.

6. What a Transformer/attention actually does

This is the architecture behind GPT-style models. The key mechanism, "self-attention," lets each token's vector get updated by looking at (and weighting) every other token's vector in the sequence — this is how the model captures relationships like "this pronoun refers to that earlier noun," which is exactly the kind of long-distance dependency your syntax skill (wh-question extraction) needs. "Attention heads" (you'll see "8 attention heads" in the spec) just means the model does this attention computation multiple times in parallel with different learned "perspectives," then combines the results.

7. Layers

The whole attention + processing step above is one "layer" (technically a transformer "block"). Stack several of these on top of each other (your spec: 4-6 layers) and each one refines the token representations further. Early layers tend to pick up simpler patterns, later layers more abstract ones — this is relevant to your "layer freeze/unfreeze" intervention (freezing early layers means "don't let basic patterns change," unfreezing later layers means "let the model reorganize its more abstract processing").

8. Parameters

"Parameters" = every single number in every `W` matrix and `b` vector across the whole model, all of them adjustable during training. "10M-30M parameters" describes total model size — small by modern standards (GPT-4 class models have hundreds of billions), which is exactly why it fits on your GPU and trains in hours instead of weeks.

9. Checkpoints

A "checkpoint" is just a saved snapshot of every weight's current value at some point during training. You'll be saving these at regular intervals so you can: (a) evaluate the model's skill accuracy and perplexity at that exact moment, (b) apply an intervention starting from that exact point, or (c) compare a "recovered" model against an "early-exposure baseline" model at matched points.

10. Perplexity (your general-capability metric)

Perplexity is a direct mathematical transform of the loss (specifically, `e^(average loss)`), measuring how "surprised" the model is by real text on average. Lower perplexity = better at predicting real language. This is your safeguard metric — if an intervention makes skill accuracy go up but perplexity gets much worse, the model got better at the one skill by

getting generally worse at language, which is the "catastrophic forgetting" pitfall, not real reopening.

11. How this maps onto the actual pipeline Fable 5 will build

- **The model** = the stack of layers described above, sized per spec (small, fits your GPU)
- **The delay scheduler** = code controlling which token sequences (BabyLM text vs. arithmetic vs. syntax data) get fed to the model at which training step — this is just deciding the *order* of your training data batches
- **The intervention module** = code that, at a specific checkpoint, either (a) overwrites the embedding layer's numbers with new random noise, (b) temporarily multiplies the LR by some factor, (c) sets `requires_grad=False` on certain layers (meaning: don't compute or apply gradients for these weights, freeze them in place), or (d) just feeds a denser burst of skill-specific data
- **The eval harness** = code that, at every checkpoint, runs the model on held-out arithmetic problems, held-out syntax minimal-pairs (BLiMP), and held-out ordinary text, and logs accuracy/perplexity numbers to a table (pandas DataFrame)

12. Terms you'll hit later (just so they're not a surprise)

- **Activation patching:** running the model normally, but at one specific layer, swapping in the internal values ("activations") from a *different* run, to test whether that layer is causally responsible for a behavior
- **Ablation:** deliberately breaking a specific part of the network (zeroing out a layer/head's output) to see how much performance drops — tells you how much that part matters
- **CKA / cosine similarity:** mathematical ways to measure how similar two sets of vectors are — used to compare whether the recovered model's internal representations resemble the original baseline's

Bottom line for you right now

You don't need to write any of this code yourself — Fable 5 handles implementation. Your job is to understand *what* each piece is doing well enough to sanity-check its output (e.g., "does this loss curve look like it's actually decreasing," "is perplexity actually being logged every checkpoint," "did it accidentally load a pretrained checkpoint instead of starting from random weights"). This doc should cover enough vocabulary for that.